

Java 网络编程期末复习题（I/O、多线程、网络编程、数据库操作）

一、单选题

1、下列关于 Java 线程的说法那些是正确的（ ）

- A、每一个 Java 线程可以看成由代码、一个真实的 CPU 以及数据三部份组成。
- B、创建线程的两种方法中，从 Thread 类中继承的创建方式可以防止出现多父类问题。
- C、Thread 类属于 java.util 程序包。
- D、以上说法无一正确。

2、运行下列程序，会产生什么结果？（ ）

```
public class X extends Thread implements Runnable{
    public void run(){
        System.out.println("this is run()");
    }
    public static void main(String args[]) {
        Thread t=new Thread(new X());
        t.start();
    }
}
```

- A、第一行会产生编译错误
- B、第六行会产生编译错误
- C、第六行会产生运行错误
- D、程序会运行和启动

3、下列选项中，用于定义接口的关键字是（ ）

- A、import
- B、implements
- C、interface
- D、protected

4、定义类头时能使用的修饰符是()。

- A) private B) static C) abstract D) protected

5、下面哪个方法不可以在任何时候被任何线程调用?()

- A、wait()
- B、sleep()
- C、yield()
- D、synchronized(this)

6、下列关于线程优先级的说法中，正确的是

- A) 线程的优先级是不能改变的
- B) 线程的优先级是在创建线程时设置的
- C) 在创建线程后的任何时候都可以设置
- D) B 和 C

7. 线程生命周期中正确的状态是
- A) 新建状态、运行状态和终止状态
 - B) 新建状态、运行状态、阻塞状态和终止状态
 - C) 新建状态、可运行状态、运行状态、阻塞状态和终止状态
 - D) 新建状态、可运行状态、运行状态、恢复状态和终止状态
8. Thread 类中能运行线程体的方法是
- A) start()
 - B) resume()
 - C) init()
 - D) run()
- 9 在程序读入字符文件时，能够以该文件作为直接参数的类是
- A) FileReader
 - B) BufferedReader
 - C) FileInputStream
 - D) ObjectInputStream
- 10) java.io 包的 File 类是
- A) 字符流类
 - B) 字节流类
 - C) 对象流类
 - D) 非流类
- 11) 下列描述中，正确的是
- A) 在 Serializable 接口中定义了抽象方法
 - B) 在 Serializable 接口中定义了常量
 - C) 在 Serializable 接口中没有定义抽象方法，也没有定义常量
 - D) 在 Serializable 接口中定义了成员方法
12. java 中用于创建文件对象的类是 ()
- A. File
 - B. Object
 - C. Thread
 - D. Frame
13. 从键盘上输入一个字符串创建文件对象，若要判断该文件对象为目录文件或数据文件，可使用下列哪个方法？ ()
- A. getPath()
 - B. getName()
 - C. isFile()
 - D. isAbsolute()
14. 下列哪个类不对直接创建对象？ ()
- A. InputStream
 - B. FileInputStream
 - C. BufferedInputStream
 - D. DataInputStream
15. 从键盘上输入多个字符时，为了避免回车换行符的影响，需要使用下列哪个流方法？ ()
- A. write()
 - B. flush()
 - C. close()
 - D. skip()
16. 以对象为单位把某个对象写入文件，则需要使用什么方法？ ()
- A. writeInt()
 - B. writeObject()
 - C. write()
 - D. writUTF()
17. 下列哪个类的方法能够直接把简单数据类型写入文件？ ()
- A. OutputStream
 - B. BufferedWriter
 - C. ObjectOutputStream
 - D. FileWriter
18. 若一个类对象能被整体写入文件，则定义该类时必须实现下列哪个接口？ ()
- A. Runnable
 - B. ActionListener
 - C. WindowsAdapter
 - D. Serializable
19. 下列哪种类型的数据能以对象的形式写入文件？ ()
- A. String
 - B. Frame
 - C. Dialog
 - D. Button
20. File 类的方法中，用于列举某目录下的子目录及文件的方法是 ()
- A. long length()
 - B. long lastModified()
 - C. String [] list()
 - D. String getName()
21. 能够以字符串为单位写入文件数据的流类是 ()
- A. FileOutputStream
 - B. FileWriter
 - C. BufferedWriter
 - C. OutputStream
22. 能够向文件输入逻辑型数据的类是： ()
- A. FileOutputStream
 - B. OutputStream
 - C. FileWriter
 - D. DataOutputStream

23. 在线程同步中，为了唤醒另一个等待的线程，使用下列方法（ ）
A. sleep() B. wait() C. notify() D. join()
24. 为了得到当前正在运行的线程，可使用下列哪个方法？（ ）
A. getName() B. Thread.currentThread() C. sleep() D. run()
25. 以下（ ）不属于线程的状态。
A. 就绪状态 B. 运行状态 C. 挂起状态 D. 独占状态
26. 当线程被创建后，其所处的状态是（ ）
A. 阻塞状态 B. 运行状态 C. 就绪状态 D. 新建状态
27. 当线程调用 start()后，其所处状态为（ ）
A. 阻塞状态 B. 运行状态 C. 就绪状态 D. 新建状态
28. Thread.sleep()方法调用后，当等待时间未到，该线程所处状态为（ ）
A. 阻塞状态 B. 运行状态 C. 就绪状态 D. 新建状态
29. 新生状态的线程其可能直接进入的状态是（ ）
A. 阻塞状态 B. 运行状态 C. 新建状态 D. 结束状态
30. Thread.sleep()方法调用后，当等待时间已到，该线程所处状态为（ ）
A. 阻塞状态 B. 运行状态 C. 就绪状态 D. 新建状态
31. 当线程因异常而退出 run()后，其所处状态为（ ）
A. 阻塞状态 B. 运行状态 C. 就绪状态 D. 结束状态
32. wait()方法首先是哪个类的方法？（ ）
A. Object B. Thread C. Runnable D. File
33. 如果我们想实现“先把要写入文件的数据先缓存到内存中，再把缓存中的数据写入文件中”的功能时，则需要使用下列哪个类？（ ）
A. FileReader B. OutputStream C. FilterOutputStream D. DataOutputStream
34. 用 read()方法读取文件内容时，判断文件结束的标记为（ ）
A. 0 B. 1 C. -1 D. 无标记
35. 以下哪个方法只对使用了缓冲的流类起作用？（ ）
A. read() B. write() C. skip() D. flush()
36. Java 提供的类 InetAddress 来进行有关 Internet 地址的操作。()
A. Socket B. ServerSocket C. DatagramSocket D. InetAddress
37. InetAddress 类中哪个方法可实现正向名称解析？（ ）
A. isReachable() B. getHostAddress() C. getHostByName() D. getByName()
38. 为了获取远程主机的文件内容，当创建 URL 对象后，需要使用哪个方法获取信息（ ）
A. getPort() B. getHost C. openStream() D. openConnection()
39. java 程序中，使用 TCP 套接字编写服务端程序的套接字类是（ ）
A. Socket B. ServerSocket C. DatagramSocket D. DatagramPacket
40. ServerSocket 的监听方法 accept()的返回值类型是（ ）
A. void B. Object C. Socket D. DatagramSocket
41. ServerSocket 的 getInetAddress()的返回值类型是（ ）
A. Socket B. ServerSocket C. InetAddress D. URL
42. 当使用客户端套接字 Socket 创建对象时，需要指定（ ）
A. 服务器主机名称和端口 B. 服务器端口和文件 C. 服务器名称和文件 D. 服务器地址和文件
43. 使用流式套接字编程时，为了向对方发送数据，则需要使用哪个方法（ ）
A. getInetAddress() B. getLocalPort() C. getOutputStream() D. getInputStream()

44. 使用 UDP 套接字通信时，常用哪个类把要发送的信息打包？（ ）
A. String B. DatagramSocket C. MulticastSocket **D. DatagramPacket**
45. 使用 UDP 套接字通信时，哪个方法用于接收数据（ ）
A. read() **B. receive()** C. accept() D. Listen()
46. 若要取得数据包的中源地址，可使用下列哪个语句（ ）？
A. getAddress() B. getPort() C. getName() D. getData()
47. 要使用 java 程序访问数据库，则必须首先与数据库建立连接，在建立连接前，应加载数据库驱动程序，该语句为（ ）
A Class.forName("sun.jdbc.odbc.JdbcOdbcDriver")
B. DriverManager.getConnection("","","")
C. Result rs= DriverManager.getConnection("","","").createStatement()
D. Statement st= DriverManager.getConnection("","","").createStatement()
48. 要使用 java 程序访问数据库，则必须首先与数据库建立连接，该语句为（ ）
A Class.forName("sun.jdbc.odbc.JdbcOdbcDriver")
B. DriverManager.getConnection("","","")
C. Result rs= DriverManager.getConnection("","","").createStatement()
D. Statement st= DriverManager.getConnection("","","").createStatement()
49. java 程序与数据库连接后，需要查看某个表中的数据，使用下列哪个语句？（ ）
A. executeQuery() B. executeUpdate() C. executeEdit() D. executeSelect()
50. java 程序与数据库连接后，需要查看某个表中的数据，使用下列哪个语句？（ ）
A. executeQuery() **B. executeUpdate()** C. executeEdit() D. executeSelect()

二、填空题

- Stdout System.out 作为 **PrintStream** 的实例，来实现 stdout，它代表的是 屏幕
- Stdin System.in 作为 **InputStream** 类的一个实例来实现 stdin，它代表的是 键盘，你可以使用其中两个成员函数，**read()** 让你从输入中读一个字节，**skip(long n)** 让你在输入中跳过 n 个字节。
- Java 中的 IO 流分为两种，一种是 字节流，另一种是 字符流，分别由四个抽象类来表示（每种流包括输入和输出两种所以一共四个）：**InputStream, OutputStream, Reader, Writer**。它们通过重载 **read()** 和 **write()** 方法定义了 6 个读写操作方法。
- File 类不是一个标准的 I/O 类，在 java.io 类库中它是唯一的 non-stream 类别，既不能读取文件内容，也不能修改文件内容，主要作用是收集档案（或目录）的相关信息，例如：**建立文件对象、修改文件名、设置文件属性、测试文件属性**
- 目录是一个包含其他文件和路径列表的 File 类。当你创建一个 **File 对象** 且它是目录时，isDirectory() 方法 **返回 true**。这种情况下，可以调用该对象的 **String [] list()** 方法来提取该目录内部其他文件和目录的列表
- 所有的输出过滤流都是抽象类_____的子类。（**FilterOutputStream**）

- 7、字符输入流 `BufferedReader` 使用了____技术。**(PrintStream)**
- 8、 **`InputStreamReader`** 负责将 `InputStream` 转化成 `Reader`，而 **`OutputStreamWriter`** 则将 `OutputStream` 转化成 `Writer`。实际上是通过 **`byte[]`**和 **`String`** 来关联。
- 9、设 `a.txt` 为当前目录下的一个文本文件，则以字符方式向该文件写数据时，需要建立的输出流通道为：**`new FileWriter("a.txt");`**
10. 随机访问文件类是 **`RandomAccessFile`**，它实现了与 **`DataInputStream`**、**`DataOutputStream`** 类同样实现的接口 `DataInput`、`DataOutput`。
11. 实现多线程的两种方式是：**1、继承 `Thread` 类；2、实现 `Runnable` 接口。**
- 12、取得当前线程的语句是：**`Thread.currentThread();`**
13. 主线程的名称是 **`main`**，默认创建的第一个子线程的名称是 **`thread-0`**
14. 可以调用 `Thread` 类的方法 **`getPriority()`** 和 **`setPriority()`** 来存取线程的优先级，线程的优先级界于 **`1 (MIN_PRIORITY)`** 和 **`10 (MAX_PRIORITY)`** 之间，**缺省是 `5 (NORM_PRIORITY)`**。
- 15.(1) 当多个线程同时运行时，会产生数据错误及其它冲突问题。Java 语言提供了线程同步控制机制，一是____锁定共享资源，使得在任何时刻只有一个线程能够访问共享资源，以保持共享资源的完整和一致，二是让互相通信的线程____运行，以保证通信的正确性。
- (2) 线程之间的通信有两种方法：一是把共享变量和方法封闭在一个类中，二是利用系统方法____和____控制线程通信。实现对共享资源互斥访问的方法是在方法声明中加入 **`synchronized`** 关键字来声明一个访问共享资源的方法，或者通过 `synchronized` 关键字来声明 **`synchronized` 块**。
- 16、守护线程一般被用于在后台为其它线程提供服务。调用方法 **`isDaemon()`** 来判断一个线程是否是守护线程，也可以调用方法 **`setDaemon()`** 将一个线程设为守护线程。
- 17、**`InetAddress`** 是用于封装 IP 地址和 DNS 的一个类。
- 18、TCP/IP 套接字是最可靠的双向流协议。等待客户端的服务器使用 **`ServerSocket` 类**，而要连接到服务器的客户端则使用 **`Socket` 类**。
- 19、java.net 包中提供了一个类 **`MulticastSocket`**，允许数据报以广播方式发送到该端口的所有客户。
20. 在 TCP/IP 协议的传输层除了 TCP 协议之外还有一个 **`UDP`** 协议。几个标准的应用层协议 HTTP, FTP, SMTP...使用的都是 **`TCP` 协议**。**`UDP` 协议**主要用于需要很强的实时交互性的场合，如**网络游戏，视频会议**等
- 21、当我们得到一个 URL 对象后，就可以通过它读取指定的 WWW 资源。这时我们将使用 URL 的方法 **`openStream()`**，其定义为：**`InputStream openStream();`**
- 22、URL 的构造方法都声明抛弃非运行时异常 **`MalformedURLException`**，因此生成 URL 对象时，我们必须要对这一例外进行处理，通常是用 **`try-catch` 语句**进行捕获。
- 23、一个 **`Statement` 对象**，可以执行多个 sql 语句以后，批量更新。这多个语句可以是 **`delete`**、**`update`**、**`insert`** 等或兼有
- 24、Java 数据库操作基本流程：**取得数据库连接、执行 sql 语句、处理执行结果、释放数据库连接。**
- 25、下列程序的功能是在监控台上每隔一秒钟显示一个字符串“Hello”，能够填写在程序中下划线位置，使程序完整并能正确运行的语句是
- ```
public class Test implements Runnable{
 public static void main(String args[]){
```

```

 Test t=new Test();
 Thread tt=new Thread(t);
 tt.start();
 }
 public void run() {
 for(;;) {
 try{
 _____;
 } catch(____e) {}
 System.out.println("Hello");
 }
 }
}

```

答：                      Thread.sleep(1000)  
                              InterruptedException

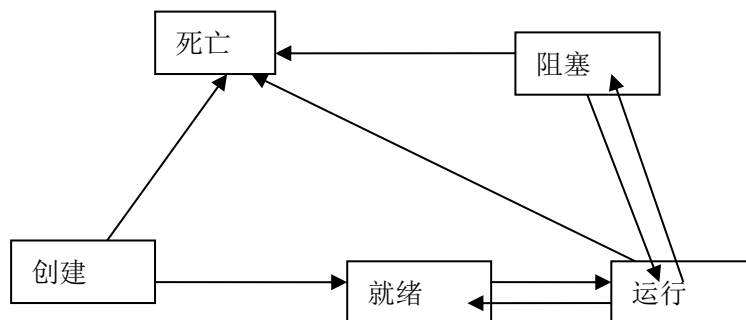
### 三、简答与程序分析、填空、设计题

#### 第一部分：简答

1、线程的生命期内包括哪些状态？

答：新建状态、就绪状态、运行状态、阻塞状态、死亡状态

见下图：各状态之间的切换关系



2、什么是线程的新建状态？

答： 线程对象已经创建，还没有在其上调用 start()方法。

3、什么是线程的可运行状态？

答：当线程有资格运行，但调度程序还没有把它选定为运行线程时线程所处的状态。当 start()方法调用时，线程首先进入可运行状态。在线程运行之后或者从阻塞、等待或睡眠状态回来后，也返回到可运行状态。

#### 4、什么是线程的死亡态？

答：当线程的 run()方法完成时就认为它死去。这个线程对象也许是活的，但是，它已经不是一个单独执行的线程。线程一旦死亡，就不能复生。 **如果在一个死去的线程上调用 start()方法，会抛出 java.lang.IllegalThreadStateException 异常。**

#### 5、调用 join()方法有何作用？

答：保证当前线程停止执行，直到该线程所加入的线程完成为止。然而，如果它加入的线程没有存活，则当前线程不需要停止。

#### 6. 现有字符串 S="hello,java!",则以此字符串生成待发送 DatagramPacket 包 dgp 的语句是：

答：

```
Byte[] str=s.getBytes();
DatagramPacket dgp=new DatagramPacket(str, str.length(),目的主机, 目的端口);
```

#### 7、介绍使用 java Socket 创建客户端 Socket 的过程。

答：try{

```
Socket socket=new Socket(目标主机,目标端口);
}catch(IOException e){
System.out.println("Error:"+e);
}
```

#### 8、**介绍使用 java ServerSocket 创建服务器端 ServerSocket 的过程。**

答：

```
ServerSocket server=null;
try {
 server=new ServerSocket(监听端口);
}catch(IOException e){
 System.out.println("can not listen to :"+e);
}
Socket socket=null;
try {
 socket=server.accept();
}catch(IOException e){
 System.out.println("Error:"+e);
}
```

#### 9、写出一种使用 java 流式套接式编程时，创建双方通信通道的语句。

答：

```
PrintStream os=new PrintStream(new BufferedOutputStream(socket.getOutputStream()));
DataInputStream is=new DataInputStream(socket.getInputStream());
PrintWriter out=new PrintWriter(socket.getOutputStream(),true);
```

```
BufferedReader in=new ButfferedReader(new InputSteramReader(Socket.getInputStream()));
```

10、对于建立功能齐全的 Socket，其工作过程包含以下四个基本的步骤：

- (1) 创建 Socket;
- (2) 打开连接到 Socket 的输入/出流;
- (3) 按照一定的协议对 Socket 进行读/写操作;
- (4) 关闭 Socket.

11、简述基于 TCP 及 UDP 套接字通信的主要区别

答：TCP，可靠，传输大小无限制，但是需要连接建立时间，差错控制开销大。

UDP，不可靠，差错控制开销较小，传输大小限制在 64K 以下，不需要建立连接。

12、写出 DatagramSocket 的常用构造方法：

答：

```
DatagramSocket ();
DatagramSocket (int prot) ;
DatagramSocket(int port, InetAddress laddr)
```

port 指明 socket 所使用的端口号，如果未指明端口号，则把 socket 连接到本地主机上一个可用的端口。laddr 指明一个可用的本地地址。

13、介绍一下 DatagramPacket 的常用构造方法：

```
DatagramPacket (byte buf[],int length);
DatagramPacket(byte buf[], int length, InetAddress addr, int port);
DatagramPacket(byte[] buf, int offset, int length);
DatagramPacket(byte[] buf, int offset, int length, InetAddress address, int port);
```

其中，buf 中存放数据报数据，length 为数据报中数据的长度，addr 和 port 指明目的地址，offset 指明了数据报的位移量。

14、在接收端接收数据报的主要语句：

```
答： byte buf[]=new byte[n];
 DatagramPacket packet=new DatagramPacket(buf, n);
 Socket.receive (packet);
```

15、写出 java 程序中用 Statement 来执行 sql 查询与更新语句

```
String sql; //表示查询或更新的操作语句;
Statement sm = cn.createStatement(); //cn 表示数据库连接
sm.executeQuery(sql); // 执行数据查询语句 (select)
sm.executeUpdate(sql); // 执行数据更新语句 (delete、update、insert、drop 等)
statement.close();
```

16. 什么叫流？流式输入输出有什么特点？

17. Java 流被分为字节流、字符流两大流类，两者有什么区别？

18. File 类有哪些构造函数和常用方法？

**16. 答：**所谓流是指同一台计算机或网络中不同计算机之间有序运动着的数据序列，Java 把这些不同来源和目标的数据都统一抽象为数据流。数据流可分为输入流和输出流，输入流代表从其他设备流入计算机的数据序列，输出流代表从计算机流向外部设备的数据序列。流式输入输出的特点是数据的获取和发送沿数据序列的顺序进行，即每一个数据都必须等待

排在它前面的数据，等前面的数据读入或送出之后才能被读写。所以流和队列一样，只能以“先进先出”的方式对其中的数据进行读写，而不能随意选择读写的位置。

**17. 答：**两者都作为 Object 类的直接子类，基本输入流（InputStream）和基本输出流（OutputStream）是处理以 8 位字节为基本单位的字节流类；Reader 和 Writer 类是专门处理 16 位字符流的类。

**18. 答：**构造函数有：

（1）public File(String pathname): 创建一个对应于参数 pathname 的 File 类对象。参数 pathname 是包含目录和文件名的字符串。如果没有文件名，则代表目录。

（2）public File(String parent, String child): 该构造函数将 pathname 分成两部分 parent 和 child，参数 parent 表示目录或文件所在路径，参数 child 表示目录或文件名称。

（3）public File(File parent, String child): 该构造函数与上面一个不同在于将 parent 的参数类型由 String 变为 File，代表 parent 是一个已经创建的 File 类文件对象（指向目录）。

常用方法有：

（1）public boolean canWrite(): 返回文件是否可写。

（2）public boolean canRead(): 返回文件是否可读。

（3）public boolean createNewFile(): 当文件不存在时创建文件。

19. 位置指针的作用是什么？RandomAccessFile 类提供了哪些方法实现对指针的控制？

**19. 答：**要实现对文件的随机读取，也就是在文件的任何位置执行读写数据的操作，需要一个指针来指定读写的位置。在创建 RandomAccessFile 类对象的同时，系统自动创建了一个指向这个文件开头的指针，当执行读写操作后，指针自动指向被读写数据之后的第一个字节处。指针初始值是 0，每读/写一个字节，指针自动增加 1。RandomAccessFile 类提供了一些控制指针移动的方法。

public long getFilePointer(); 获取当前指针指向文件的位置。

public void seek(long pos); 将指针移动到参数 pos 指定的位置。

public int skipBytes(int n); 指针从当前位置向后移动 n 个字节位置，并返回指针实际移动的字节数。

## 第二部分：分析程序功能或完成程序

2、利用文件输入输出流编写一个实现文件拷贝的程序，源文件名和目标文件名通过命令行参数传入。

答：

```
import java.io.*;
class CopyFile{
public static void main(String[] args) {
String file1,file2 ;
int ch = 0 ;
try {
file1=args[0];
file2=args[1];
FileInputStream fis = new FileInputStream(file1);
```

```

FileOutputStream fos=new FileOutputStream(file2);
while ((ch=fis.read())!=-1)
 fos.write(ch);
fis.close();
fos.close();
}
catch(FileNotFoundException e){
 System.out.println("源文件:未找到!");
} catch(ArrayIndexOutOfBoundsException e){
 System.out.println("缺少运行参数! ");
 System.exit(-1);
}
catch (IOException e){
 System.out.println(e.toString());
}
}
}
}

```

4、编写一个程序，从键盘输入一串字符，统计这串字符中英文字母、数字、其他符号的字符数。

4. 答：

```

import java.io.*;
public class Count{
 public static void main(String[] args)
 {
 int x=0,y=0,z=0;
 int ch;
 try{
 while((ch=System.in.read())!='\r'){
 if(ch>='A'&&ch<='Z' || ch>='a'&&ch<='z')
 x++;
 else if(ch>='0'&&ch<='9')
 y++;
 else
 z++;
 }
 } catch(IOException e){
 System.out.println(e.toString());
 }
 System.out.println("英文字母: "+x);
 System.out.println("数字字符: "+y);
 System.out.println("其它字符: "+z);
 }
}

```

5. 编写一个程序，从键盘输入一串字符，从屏幕输出并将其存入 a.txt 文件中。

5. 答：

```
import java.io.*;
public class InFile{
public static void main(String[] args)
{
int ch;
try{
FileOutputStream out=new FileOutputStream("a.txt");
while((ch=System.in.read())!='\r'){
System.out.write(ch);
out.write(ch);
}
out.close();
System.out.write('\n');
}catch(IOException e){
System.out.println(e.toString());
}
System.out.println("输出至文件完毕！");
}
}
```

8、下列程序向屏幕输出文本文件内容：请完成所缺少的代码

```
import java.io.*;
class readtxt
{
public static void main(String[] args) {

try{
FileReader fl = new FileReader("u.txt");
BufferedReader bfl =new BufferedReader(fl);

String line;
line = bfl.readLine();
while(line!=null) //判断所读的一行内容是否为空；
{
System.out.println(line);

line = bfl.readLine(); //继续读入下一行
}
bfl.close(); //关闭文件的输入流；
}catch(IOException e) {e.printStackTrace(); }
}}
```

10、请编写 java 程序，访问 <http://www.tirc1.cs.tsinghua.edu.cn> 所在的主页文件

```
答： public class URLReader {
 public static void main(String[] args) throws Exception { /声明抛出所有例外
 URL tirc = new URL("http://www.tirc1.cs.tsinghua.edu.cn/"); //构建一 URL 对象
 BufferedReader in = new BufferedReader(new InputStreamReader(tirc.openStream()));
 //使用 openStream 得到一输入流并由此构造一个 BufferedReader 对象
 String inputLine;
 while ((inputLine = in.readLine()) != null) //从输入流不断的读数据，直到读完为止
 System.out.println(inputLine); //把读入的数据打印到屏幕上
 in.close(); //关闭输入流
 }
}
```

11、从键盘上输入主机名称，编写类似 ping 的程序，测试连接效果。

```
import java.net.*;
import java.io.*;

class Ping {
 public static void main(String[] args)
 { System.out.println("请输入 ping 主机名称或地址: \n");

 try {
 String host = keyreadline();

 InetAddress ip = InetAddress.getByName(host);
 long t1 = System.currentTimeMillis();
 if(ip.isReachable(5000))
 { long t2 = System.currentTimeMillis();
 System.out.println("\nReply from " + ip.getHostAddress() + " time<=" + (t2-t1) + "ms");
 System.out.println();
 }
 else { System.out.println("Request timed out.");
 } }
 catch (IOException e)
 { System.out.println("Request timed out."); }
 }

 private static String keyreadline() {
 BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
 String str = null;
 try {
 str = br.readLine();
 }
```

```

 } catch (IOException e) {
 e.printStackTrace();
 }
 return str;
 }
}

```

12、设服务器端程序监听端口为 8629, 当收到客户端信息后, 首先判断是否是“BYE”, 若是, 则立即向对方发送“BYE”, 然后关闭监听, 结束程序。若不是, 则在屏幕上输出收到的信息, 并由键盘上输入发送到对方的应答信息。请编写程序完成此功能。

```

import java.net.*;
import java.io.*;

class Server{
 public Server() {
 try {
 ServerSocket ss = new ServerSocket(8629);
 System.out.println("服务器端已运行, 等待客户端连接: ");
 Socket socket = ss.accept(); //接收客户端的连接请求;
 BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
 PrintWriter out = new PrintWriter(socket.getOutputStream(),true);
 BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
 String line;
 while(true)
 {line = (in.readLine().trim());
 if(line.equals("BYE"))
 { out.println("BYE");
 out.close();
 in.close();
 socket.close();
 br.close();
 ss.close();
 break;}
 System.out.println("客户输入为 :"+ line);
 System.out.println("请输入回答:");
 line=br.readLine(); //从键盘上输入信息;
 out.println(line); //向对方发送信息
 }
 }
 catch (IOException e){}
 }
 public static void main(String[] args) throws Exception{
 new Server();
 }
}

```

13、TCP 客户端需要向服务器端 8629 发出连接请求，与服务器进行信息交流，当收到服务器发来的是“BYE”时，立即向对方发送“BYE”，然后关闭连接，否则，继续向服务器发送信息。

```
import java.io.*;
import java.net.*;

class Client2{
public Client2()
{ try
{String s;
Socket socket = new Socket("服务器地址", 8629);
BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
PrintWriter out = new PrintWriter(socket.getOutputStream(),true);
BufferedReader line = new BufferedReader(new InputStreamReader(System.in));
while(true)
{ System.out.println("请向服务器输出一条字符串：");
s=line.readLine();
out.println(s); //向服务器发出信息;
s=in.readLine().trim(); //收到服务器发来的信息;
System.out.println("服务器返回的信息是：");
System.out.println(s);
if(s.equals("BYE"))
{ out.println("BYE");

line.close();
out.close();
in.close();
socket.close(); break;}
}
}
catch (IOException e){}
}

public static void main(String[] args) {
new Client2();
}}
```

14、UDP 接收端

```
import java.io.*;
import java.net.*;

class Server {
public static void main(String[] args) {
new UDPServer() ; }
}
```

```

class UDPServer {
 public UDPServer() {
 try {
 DatagramSocket socket = new DatagramSocket(1986); //建立一个 Socket 连接
 System.out.println("服务器 IP: " + InetAddress.getLocalHost().getHostAddress()); //显示服务器信息
 while (true) {
 // 接收
 byte[] instr = new byte[200];
 DatagramPacket receive = new DatagramPacket(instr, instr.length);
 socket.receive(receive); //接收客户机发过来的数据
 String str = new String(receive.getData()); //提取数据
 if(str.trim().equals("BYE"))
 {socket.close();break;}
 System.out.println("收到客户端消息: " + str.trim()); //打印客户机消息
 // 发送
 System.out.print("服务器发送消息: ");
 str = new BufferedReader(new InputStreamReader(System.in)).readLine(); //得到输入的数据
 byte[] outstr = new byte[str.length()];
 outstr = str.getBytes();
 DatagramPacket send = new DatagramPacket(outstr,outstr.length, receive.getAddress(),
receive.getPort()); //向客户机发送数据
 socket.send(send); //执行发送
 }
 } catch (Exception e) {
 //错误处理
 //e.printStackTrace();
 }
 }
}

```

## 15、UDP 发送端

```

import java.io.*;
import java.net.*;
class Client {
 public static void main(String[] args) {
 new UDPClient();
 }
}
class UDPClient {
 public UDPClient() {

```

```

System.out.print("请输入服务器的地址: ");
String serverip = null;
try {
 serverip = new BufferedReader(new InputStreamReader(System.in)).readLine(); //取得输入的服务器地址
} catch (IOException e1) {
 //错误处理
 //e1.printStackTrace();
}
try {
 DatagramSocket socket = new DatagramSocket(); //建立 Socket
 while (true) {
 // 发送
 System.out.print("客户端发送消息: ");
 String send = new BufferedReader(new InputStreamReader(System.in)).readLine(); //读入客户机输入的消息
 byte[] outstr = new byte[send.length()];
 outstr = send.getBytes();
 InetAddress address = InetAddress.getByName(serverip);
 DatagramPacket packet = new DatagramPacket(outstr, outstr.length, address, 1986); //向服务器发送数据报
 socket.send(packet); //发送数据报
 // 接收
 byte[] instr = new byte[200];
 packet = new DatagramPacket(instr, instr.length); //接收数据报
 socket.receive(packet); //接收数据
 String out = new String(packet.getData()); //取出信息
 if(out.trim().equals("BYE"))
 {socket.close();break;}
 System.out.println("收到服务器消息: " + out.trim()); //打印服务器发过来的信息
 }
} catch (Exception e) {
 // 错误处理
 // e.printStackTrace();
}
}
}

```

16、设 access 数据库中有 student 表，表中存放学生学号，姓名两个字段，请编写程序输出表中所有记录信息。（假定已经配置好数据源为：mydb11）

```

import java.sql.*;
class access{
public static void main(String args[]){
 Connection con;

```

```
Statement sql; //声明 Statement 对象
ResultSet rs;
try{
 Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
}
catch(ClassNotFoundException e){
 System.out.println(""+e);
}
try{
 con=DriverManager.getConnection("jdbc:odbc:mydb11","","");
 sql=con.createStatement();
 rs=sql.executeQuery("Select * FROM mytable");
 while(rs.next()){
 String num=rs.getString(1); //获得数据库第一列
 String name=rs.getString(2);
 System.out.print("学号:"+num); //输出信息
 System.out.print(" 姓名:"+name);
 System.out.println();
 }
 con.close();
}
catch(SQLException el){}
```